Finite State Machine - Elevator Controller FPGA Implementation (Honors)

By: Mina George and Eric Hu

ECE 2610 - Sec 001

December 1, 2025

Table of Contents:

Abstract: 3
Introduction: 4
Design: 6
Results: 8
Verilog Code: 9
State and Timing Diagrams: 16
FPGA Implementation: 18
Conclusion: 25

Abstract:

A finite state machine allows for the storing of states while providing outputs based on the current state the machine is at and/or the input of the machine. They are a fundamental concept in the creation of sequential logic circuits and utilize combinations of both combinational and sequential logic. The objective of this project was to implement an elevator controller on an FPGA board utilizing Verilog code. By creating the state diagram, Verilog code that simulates the various states could be utilized to construct a basic timing diagram for the system. Once this was completed, the rest of the modules were created, which allowed for the design to be visually implemented and uploaded onto the FPGA board.

By incorporating a clock divider to lower the clock frequency, a time-multiplexing unit for the seven-segment output displays, and a finite state machine that generated the outputs, a top module instantiating all codes together allowed for the creation of the system. Therefore, utilizing the pushbuttons as inputs for the elevator floor as well as switches for the emergency and reset state, the outputs could visually be displayed on the FPGA board using the LEDs and the Seven-Segment Displays.

Introduction:

The objective of this project was to construct a finite state machine that simulated an elevator controller. The finite state machine can be designed in Verilog and will allow for the implementation of the elevator controller onto an FPGA board. The pushbuttons on the FPGA board should correspond to the inputs for the desired elevator floor, and the seven-segment displays should show the desired elevator floor, as well as the current floor. Furthermore, an LED would be used to indicate that the user has flipped the emergency switch, indicating that it is currently in an emergency state. Lastly, the reset state will be controlled using one of the switches as well, forcing the current floor to go to ground, which was set as the initial state.

The system for the elevator controller should initially begin on the ground floor; however, when the user inputs a floor using the pushbutton, the elevator should go to the desired floor and update the displays accordingly. Both the desired and current floor will update on the seven-segment displays. If the user enters the current floor, the elevator will stay at the current floor without changing the values on the displays, as the current state of the elevator is stored within the finite state machine.

Within the concept of finite state machines, the distinction between a Moore and Mealy machine is critical, as this decides how the problem will be approached in all aspects. The type of machine impacts the state diagram, simulations, and the main code; therefore, it is critical that an adequate machine is chosen. In the case of Moore machines, the output depends only on the current state of the system, whereas the outputs of a Mealy machine depend on both the current state and

the primary inputs. The elevator has two outputs; therefore, it has two possibilities for the type of machine. In fact, both outputs are of a different type; therefore, the elevator has both elements of a Moore and Mealy machine.

One seven-segment display is responsible for showing the current floor/state of the elevator. This output is a Moore machine, as the output only depends on the current floor of the elevator, which is the state it currently resides in. The other seven-segment display is responsible for showing the next floor of the elevator whenever a floor button different from the current floor is selected via the pushbuttons. This display stays at the current floor number if no other floor is selected and is the desired floor once one is selected. Due to relying on both the floor (when no other floor is selected) and an input (the newly selected floor), this output is a Mealy machine.

Design:

To create this system, a state diagram that shows the current states as well as how the inputs relate to the next states and outputs must be constructed. Using this state diagram, a Verilog module that models this finite state machine, along with a testbench code, can be created to simulate the system. Once this is completed, a control unit that lowers the FPGA's internal clock to 1Hz, as well as a display unit which displays the desired values onto the seven-segment displays using time-division multiplexing, was designed.

In terms of the FSM code, this module controls the state of the machines and handles essential logic. The bulk of this code consists of case statements, which reflect and describe each state of the system. The code for the Moore machine output simply displays the floor number of whatever the current state is. However, the Mealy machine output is slightly more complex. This requires that each state has a path to every other state, since an elevator should be able to go from any floor to any other floor. This means that if-else statements were the most logical to use, one in each case statement for each state. Within this project, one extra floor was added to complete the honors requirement, and the other was for extra credit; therefore, combining for a total of five different floor options.

Once all the supplementary codes had been created, a top module that incorporates and instantiates all the codes so that they can be used in one system was created. This allowed for the creation and completion of the finite state machine system that accurately modelled an elevator

controller. Lastly, a constraint file that indicates which inputs and outputs correspond to each component on the FPGA board was designed. This allowed for the inputs to be controlled with the respective pushbuttons, the reset and emergency states to be controlled with the respective switches, the LED display to correspond to the emergency state, and the seven-segment displays to show the respective outputs of the elevator current and desired floors.

Results:

After creating the various Verilog modules, which consisted of clock frequency division, seven-segment display control, and the finite state machine structure, the implementation of the system was accurately modelled on the FPGA board. After specifying the components through the constraint file, the pushbuttons were utilized as the elevator floor inputs, the switches were used to control the emergency and reset states, and lastly, the LED and seven-segment displays showed the outputs based on the emergency state and the current and desired floor.

In this project, 5 floors were implemented that could be reached no matter the current state of the elevator. On the seven-segment displays, the current and desired floors were shown, as well as animation visual effects on the unused displays to appeal to the user. Various images of the timing diagram, state diagram, and the FPGA board during different scenarios are shown below, as well as the Verilog code that was designed to model the system:

Verilog Code:

```
1   `timescale 1ns / 1ps
2
3   module FSM(
4       input wire clk,
5       input wire reset,
6       input wire g_f,
7       input wire f_f,
8       input wire s_f,
9       input wire third_f,
10      input wire fourth_f,
11      input wire emerg_in,
12      output reg [3:0] disp_1, // Destination Floor
13      output reg [3:0] disp_2, // Current/From Floor
14      output reg [3:0] disp_3,
15      output reg [3:0] disp_4,
16      output reg emerg_out
17  );
18      reg [2:0] y;
19      parameter [2:0] Ground = 3'b000, First = 3'b001, Second = 3'b010, Third = 3'b101, Fourth = 3'b110, Emergency = 3'b011;
20
21      always @(posedge clk) begin
22          if (reset) begin
23              y <= Ground;
24              emerg_out <= 0;
25              disp_1 <= 4'b0000;
26              disp_2 <= 4'b0000;
27              disp_3 <= 4'b0000;
28              disp_4 <= 4'b0000;
29          end
30          else begin
31              // Global check: If emergency is pulled, override everything
32              // (Moving this outside the case statement is cleaner logic)
33              if (emerg_in == 1 && y != Emergency) begin
34                  y <= Emergency;
35                  disp_1 <= 4'b1110;
36                  disp_2 <= 4'b1110;
37                  disp_3 <= 4'b1110;
```

Figure 1: FSM Code Part 1

```
            disp_3 <= 4'b1110;
            disp_4 <= 4'b1110;
            emerg_out <= 1;
        end
        else begin
            case (y)
                Ground: begin
                    disp_3 <= 4'b0000; disp_4 <= 4'b0000; emerg_out <= 0;
                    if (g_f) begin y <= Ground; disp_1 <= 4'b0000; disp_2 <= 4'b0000; end
                    else if (f_f) begin y <= First; disp_1 <= 4'b0001; disp_2 <= 4'b0000; end
                    else if (s_f) begin y <= Second; disp_1 <= 4'b0010; disp_2 <= 4'b0000; end
                    else if (third_f) begin y <= Third; disp_1 <= 4'b0011; disp_2 <= 4'b0000; end   // Added...
                end

                First: begin
                    disp_3 <= 4'b0000; disp_4 <= 4'b0000; emerg_out <= 0;
                    if (f_f) begin y <= First; disp_1 <= 4'b0001; disp_2 <= 4'b0001; end
                    else if (g_f) begin y <= Ground; disp_1 <= 4'b0000; disp_2 <= 4'b0001; end
                    else if (s_f) begin y <= Second; disp_1 <= 4'b0010; disp_2 <= 4'b0001; end
                    else if (third_f) begin y <= Third; disp_1 <= 4'b0011; disp_2 <= 4'b0001; end   // Added
                    else if (fourth_f) begin y <= Fourth; disp_1 <= 4'b0100; disp_2 <= 4'b0001; end // Added
                    else begin disp_1 <= 4'b0001; disp_2 <= 4'b0001; end
                end

                Second: begin
                    disp_3 <= 4'b0000; disp_4 <= 4'b0000; emerg_out <= 0;
                    if (s_f) begin y <= Second; disp_1 <= 4'b0010; disp_2 <= 4'b0010; end
                    else if (g_f) begin y <= Ground; disp_1 <= 4'b0000; disp_2 <= 4'b0010; end
                    else if (f_f) begin y <= First; disp_1 <= 4'b0001; disp_2 <= 4'b0010; end
                    else if (third_f) begin y <= Third; disp_1 <= 4'b0011; disp_2 <= 4'b0010; end   // Added
                    else if (fourth_f) begin y <= Fourth; disp_1 <= 4'b0100; disp_2 <= 4'b0010; end // Added
                    else begin disp_1 <= 4'b0010; disp_2 <= 4'b0010; end
                end

                Third: begin
                    disp_3 <= 4'b0000; disp_4 <= 4'b0000; emerg_out <= 0;
                    if (third_f) begin y <= Third; disp_1 <= 4'b0011; disp_2 <= 4'b0011; end
```

Figure 2: FSM Code Part 2

```
75                      if (third_f) begin y <= Third; disp_1 <= 4'b0011; disp_2 <= 4'b0011; end
76                      else if (g_f) begin y <= Ground; disp_1 <= 4'b0000; disp_2 <= 4'b0011; end // Added
77                      else if (f_f) begin y <= First; disp_1 <= 4'b0001; disp_2 <= 4'b0011; end  // Added
78                      else if (s_f) begin y <= Second; disp_1 <= 4'b0010; disp_2 <= 4'b0011; end // Added
79                      else if (fourth_f) begin y <= Fourth; disp_1 <= 4'b0100; disp_2 <= 4'b0011; end
80                      else begin disp_1 <= 4'b0011; disp_2 <= 4'b0011; end
81                  end
82
83                  Fourth: begin  // This block was completely missing
84                      disp_3 <= 4'b0000; disp_4 <= 4'b0000; emerg_out <= 0;
85                      if (fourth_f) begin y <= Fourth; disp_1 <= 4'b0100; disp_2 <= 4'b0100; end
86                      else if (g_f) begin y <= Ground; disp_1 <= 4'b0000; disp_2 <= 4'b0100; end
87                      else if (f_f) begin y <= First; disp_1 <= 4'b0001; disp_2 <= 4'b0100; end
88                      else if (s_f) begin y <= Second; disp_1 <= 4'b0010; disp_2 <= 4'b0100; end
89                      else if (third_f) begin y <= Third; disp_1 <= 4'b0011; disp_2 <= 4'b0100; end
90                      else begin disp_1 <= 4'b0100; disp_2 <= 4'b0100; end
91                  end
92
93                  Emergency: begin
94                      // Stay here until emergency button is released
95                      if (emerg_in == 0) begin
96                          y <= Ground;
97                          emerg_out <= 0;
98                          disp_1 <= 4'b0000; disp_2 <= 4'b0000;
99                          disp_3 <= 4'b0000; disp_4 <= 4'b0000;
100                     end
101                     else begin
102                         y <= Emergency;
103                         emerg_out <= 1;
104                     end
105                 end
106
107                 default: begin
108                     y <= Ground;
109                     emerg_out <= 0;
110                     disp_1 <= 4'b0000; disp_2 <= 4'b0000;
111                 end
```

Figure 3: FSM Code Part 3

```
1  `timescale 1ns / 1ps
2
3  module hex_mux
4  (
5      input wire clk, reset,
6      input wire [3:0] hex3, hex2, hex1, hex0, // hex digits
7      // unused_mask removed - logic is now internal
8      output reg [3:0] an,                    // enable 1-out-of-4, active low
9      output reg [6:0] seg                    // led segments a-g
10 );
11
12     // N needs to be larger now.
13     // Bits [17:16] control multiplexing (fast).
14     // Bits [25:23] control animation speed (slow).
15     localparam N = 26;
16
17     // Internal signals
18     reg [N-1:0] q_reg;
19     wire [N-1:0] q_next;
20
21     reg [3:0] hex_in;
22     reg current_digit_unused; // Flag to check if current digit is in "animation mode"
23
24     // N-bit counter
25     always @(posedge clk or posedge reset) begin
26         if (reset)
27             q_reg <= 0;
28         else
29             q_reg <= q_next;
30     end
31
32     assign q_next = q_reg + 1;
33
34     // ---------------------------------------------------------
35     // MUX CONTROL
36     // Uses bits [17:16] for ~800Hz refresh rate (standard muxing)
37     // ---------------------------------------------------------
```

Figure 4: Hex Mux Code Part 1

```
37      // ----------------------------------------------------------
38      always @(*) begin
39          // Default all digits off
40          an = 4'b1111;
41          current_digit_unused = 0; // Default to false
42
43          case(q_reg[17:16])
44              2'b00: begin
45                  an[0] = 0;              // rightmost digit (disp_1)
46                  hex_in = hex0;
47                  current_digit_unused = 0; // Always show number
48              end
49              2'b01: begin
50                  an[1] = 0;              // 2nd digit (disp_2)
51                  hex_in = hex1;
52                  current_digit_unused = 0; // Always show number
53              end
54              2'b10: begin
55                  an[2] = 0;              // 3rd digit (disp_3)
56                  hex_in = hex2;
57                  // Animate ONLY if it is NOT showing 'E' (Emergency)
58                  // This allows the Emergency state to still override the animation
59                  if (hex_in != 4'hE) current_digit_unused = 1;
60                  else current_digit_unused = 0;
61              end
62              2'b11: begin
63                  an[3] = 0;              // leftmost digit (disp_4)
64                  hex_in = hex3;
65                  // Animate ONLY if it is NOT showing 'E'
66                  if (hex_in != 4'hE) current_digit_unused = 1;
67                  else current_digit_unused = 0;
68              end
69          endcase
70      end
71
72      // ----------------------------------------------------------
73      // ANIMATION & SEGMENT DECODER
```

Figure 5: Hex Mux Code Part 2

```
72      // ------------------------------------------------------
73      // ANIMATION & SEGMENT DECODER
74      // ------------------------------------------------------
75      always @(*) begin
76          if (current_digit_unused) begin
77              // --- COOL ANIMATION MODE ---
78              // Uses bits [25:23] for slow ~6Hz animation speed
79              // Pattern: Circle (A -> B -> C -> D -> E -> F)
80              case (q_reg[25:23])
81                  3'b000: seg = 7'b0111111; // Seg A ON
82                  3'b001: seg = 7'b1011111; // Seg B ON
83                  3'b010: seg = 7'b1101111; // Seg C ON
84                  3'b011: seg = 7'b1110111; // Seg D ON
85                  3'b100: seg = 7'b1111011; // Seg E ON
86                  3'b101: seg = 7'b1111101; // Seg F ON
87                  default: seg = 7'b0111111; // Loop back to A
88              endcase
89          end
90          else begin
91              // --- STANDARD HEX DISPLAY MODE ---
92              case(hex_in)
93                  4'h0 : seg = 7'b1000000;
94                  4'h1 : seg = 7'b1111001;
95                  4'h2 : seg = 7'b0100100;
96                  4'h3 : seg = 7'b0110000;
97                  4'h4 : seg = 7'b0011001;
98                  4'h5 : seg = 7'b0010010;
99                  4'h6 : seg = 7'b0000010;
00                  4'h7 : seg = 7'b1111000;
01                  4'h8 : seg = 7'b0000000;
02                  4'h9 : seg = 7'b0010000;
03                  4'hA : seg = 7'b0001000;
04                  4'hB : seg = 7'b0000011;
05                  4'hC : seg = 7'b1000110;
06                  4'hD : seg = 7'b0100001;
07                  4'hE : seg = 7'b0000110;
08                  4'hF : seg = 7'b0001110;
```

Figure 6: Hex Mux Code Part 3

```
104                 4'hB : seg = 7'b0000011;
105                 4'hC : seg = 7'b1000110;
106                 4'hD : seg = 7'b0100001;
107                 4'hE : seg = 7'b0000110;
108                 4'hF : seg = 7'b0001110;
109                 default: seg = 7'b1111111;
110             endcase
111         end
112     end
113
114 endmodule
```

Figure 7: Hex_Mux Code Part 4

```
1  `timescale 1ns / 1ps
2
3  module cntr_unit(
4      input wire clk,
5      output reg out
6  );
7
8      // Signal declaration
9      reg [31:0] r_reg;
10     wire [31:0] r_next;
11     // Register - counter
12     always @(posedge clk) begin
13         r_reg <= r_next;
14     end
15     // Next state logic - counts from 0 to 49,999,999 (50M cycles for 1 Hz at 50 MHz)
16     // For 1 Hz clock: 50,000,000 - 1 = 49,999,999
17     assign r_next = (r_reg == 49_999_999) ? 0 : r_reg + 1;
18     // Output logic - generates a proper clock signal (toggles every 50M cycles)
19     always @(posedge clk) begin
20         if (r_reg == 49_999_999)
21             out <= ~out;  // Toggle to create 50% duty cycle clock
22     end
23
24 endmodule
```

Figure 8: Control Unit Code

State and Timing Diagrams:

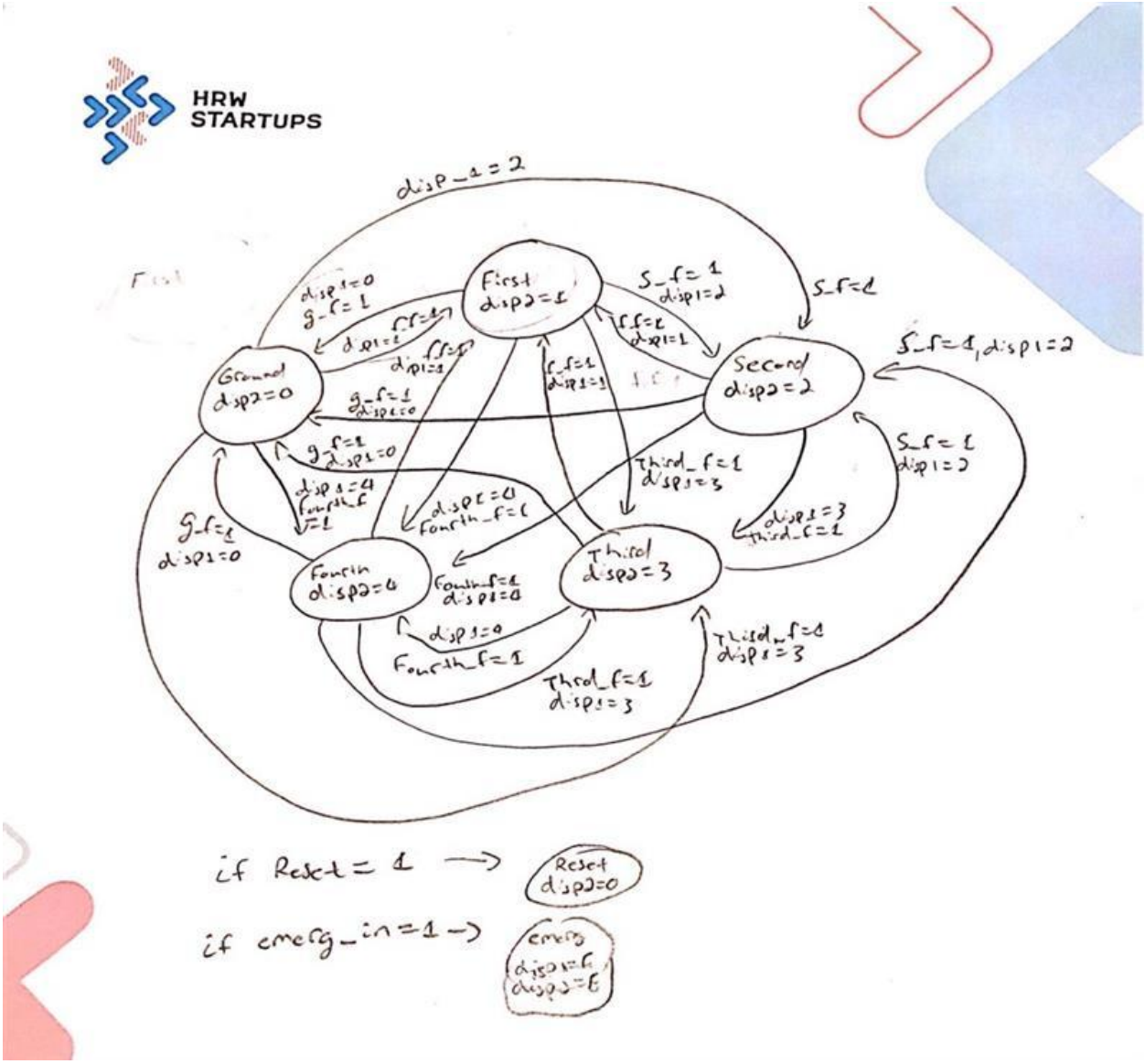


Figure 9: State Diagram of Final Code

This state diagram models the system that was constructed, as there are five different states that the system could reside in, and each state needs to have a path to every other state, thus making the diagram quite crowded. The reset and emergency states were placed separately to reduce the complexity of the diagram. Since the output and state of these two states only depend on the input and nothing else, they could be placed off to the side without the need for incorporating them within the state diagram directly.

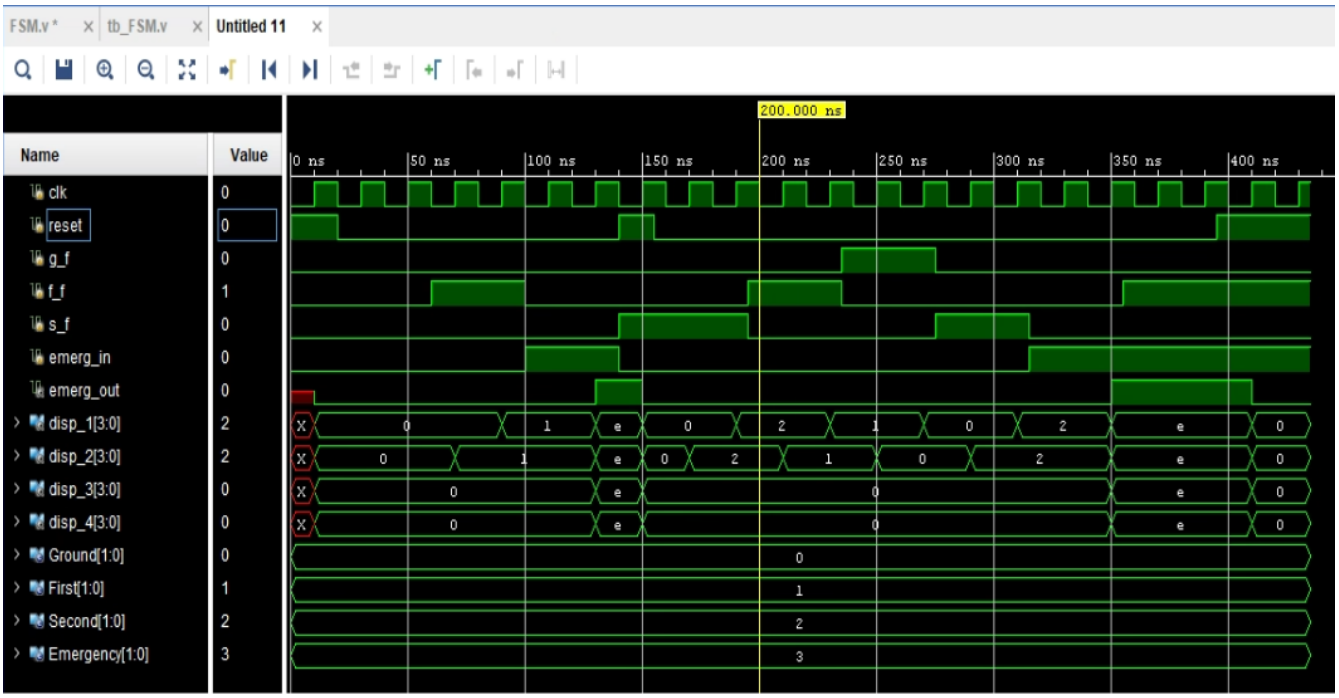


Figure 10: Timing Diagram of Source Code

The timing diagram was described in detail in the previous deliverable, including the various scenarios that it depicts. On a surface level, it's clear that each floor button can send the elevator to that corresponding floor, with one of the seven-segment displays displaying that floor first, then once the elevator has arrived at the floor, the other display changes. The reset also clearly sets the floor to zero/ground when the input value is high. Finally, the emerg_in input causes the outputs to display "E"; the emergency state is held until emerg_in is switched off. The image below shows a timing diagram for the final FSM, which includes all the extra floors.

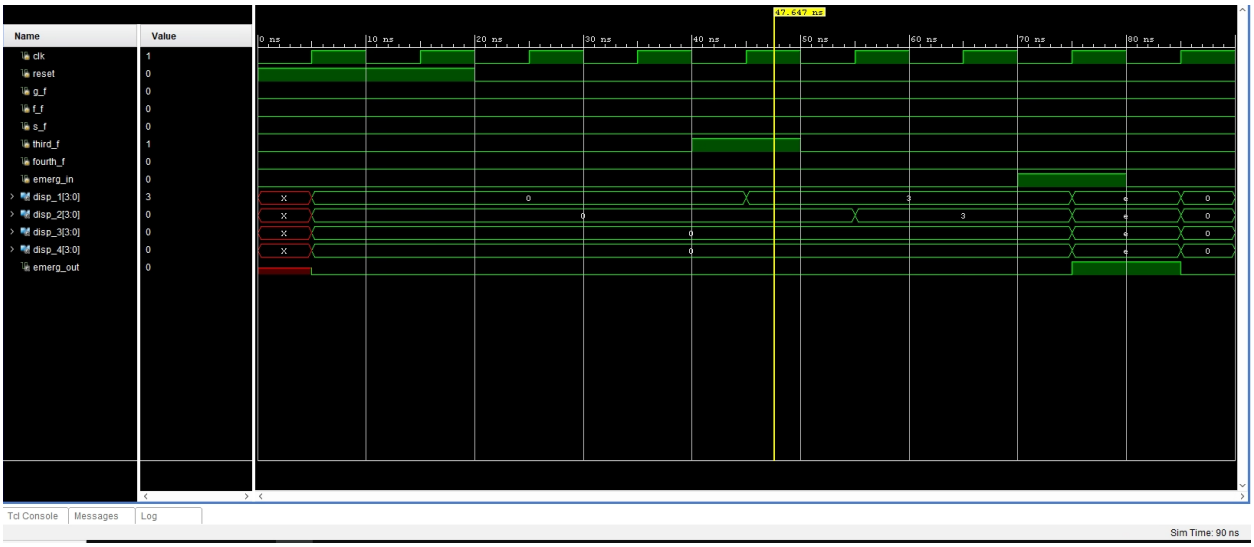


Figure 11: Timing Diagram with Extra Floors

FPGA Implementation:

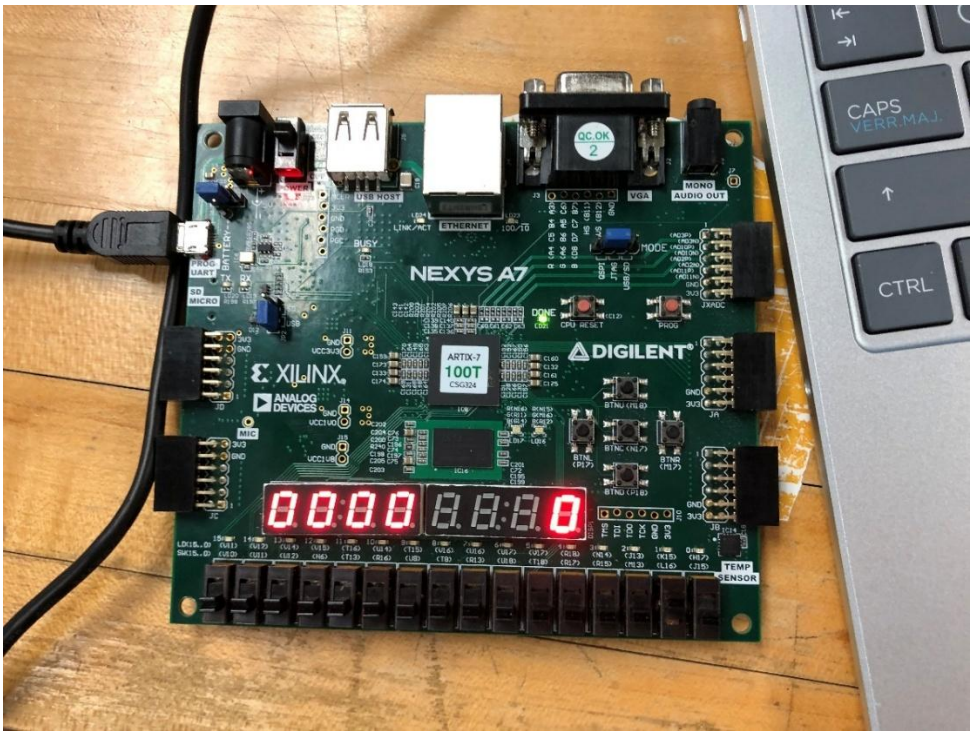


Figure 12: FPGA after the reset switch is flipped

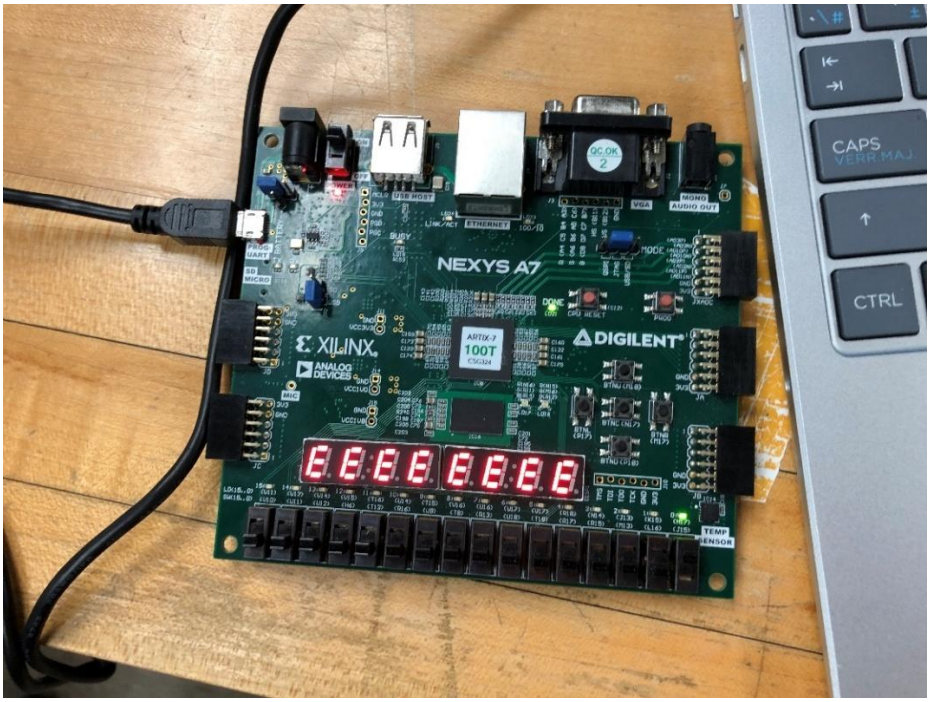


Figure 13: FPGA in Emergency State

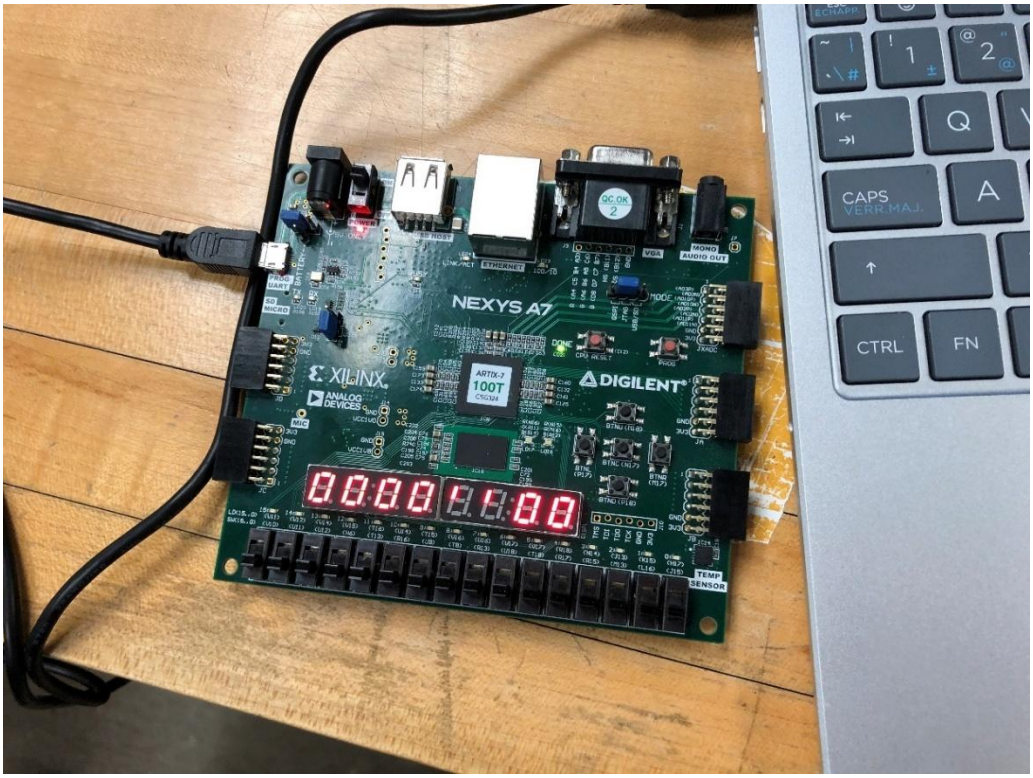


Figure 14: FPGA on ground floor with no other floors selected

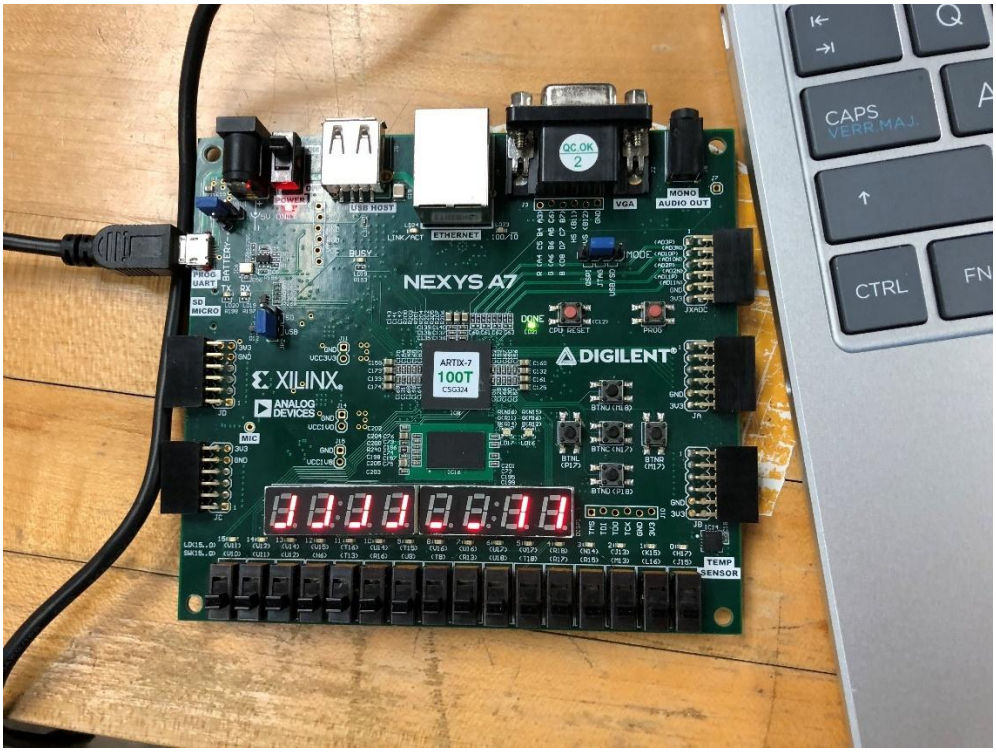


Figure 15: FPGA on First floor with no other floors selected

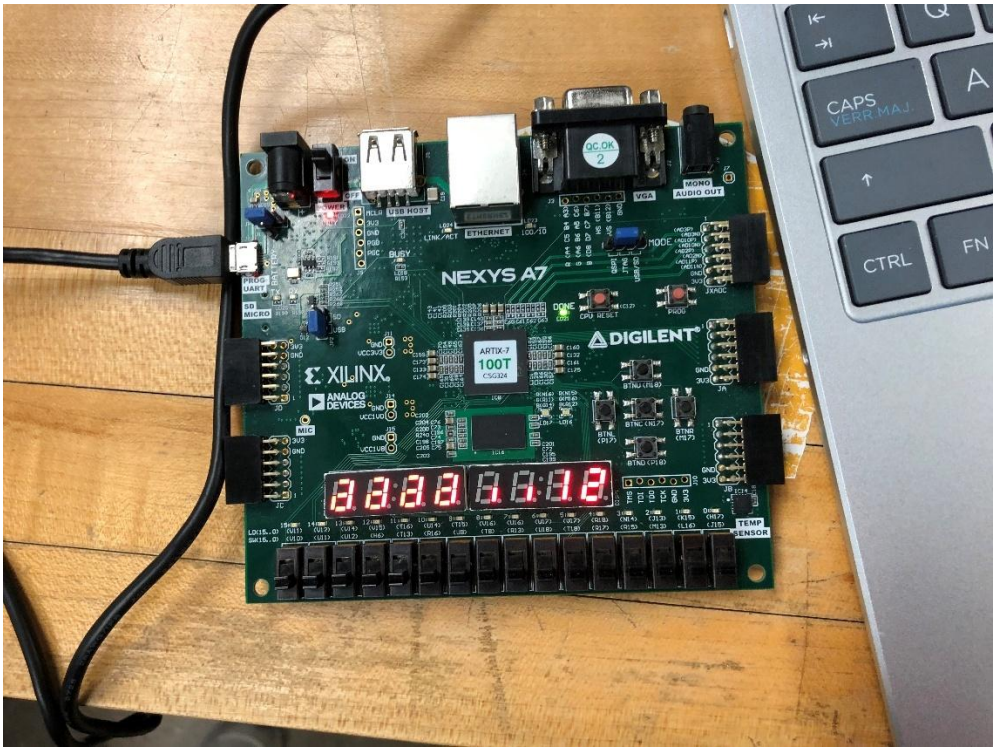


Figure 16: FPGA on first floor with second floor button just selected

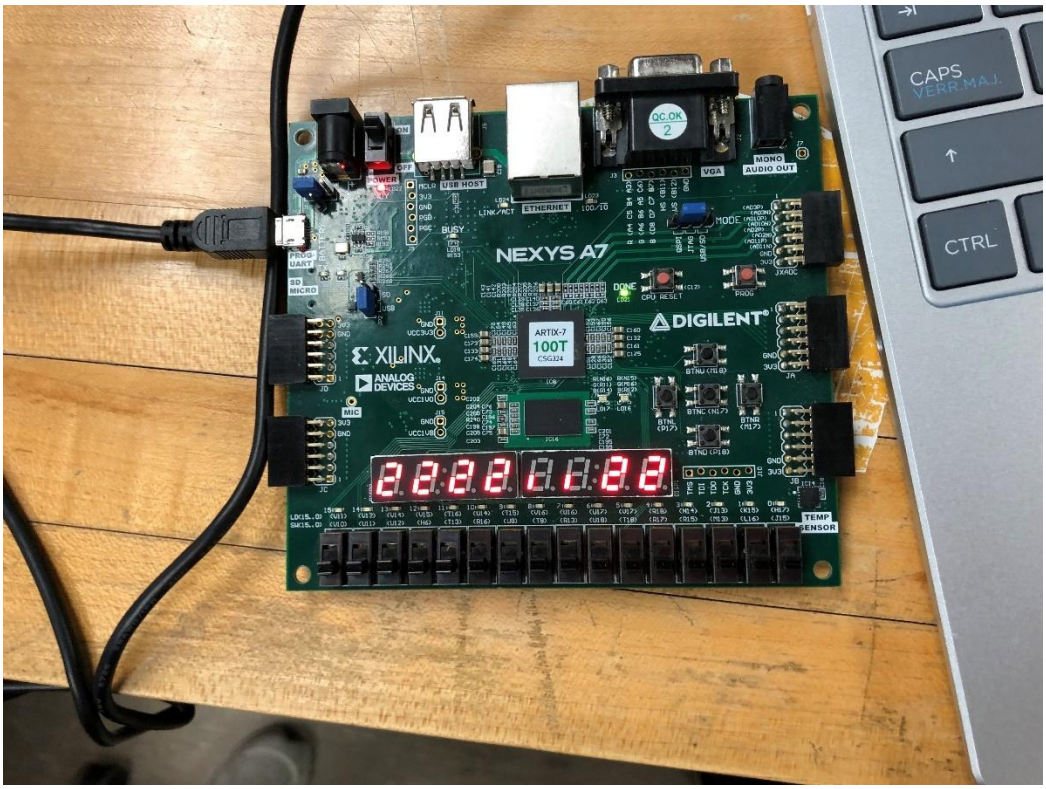
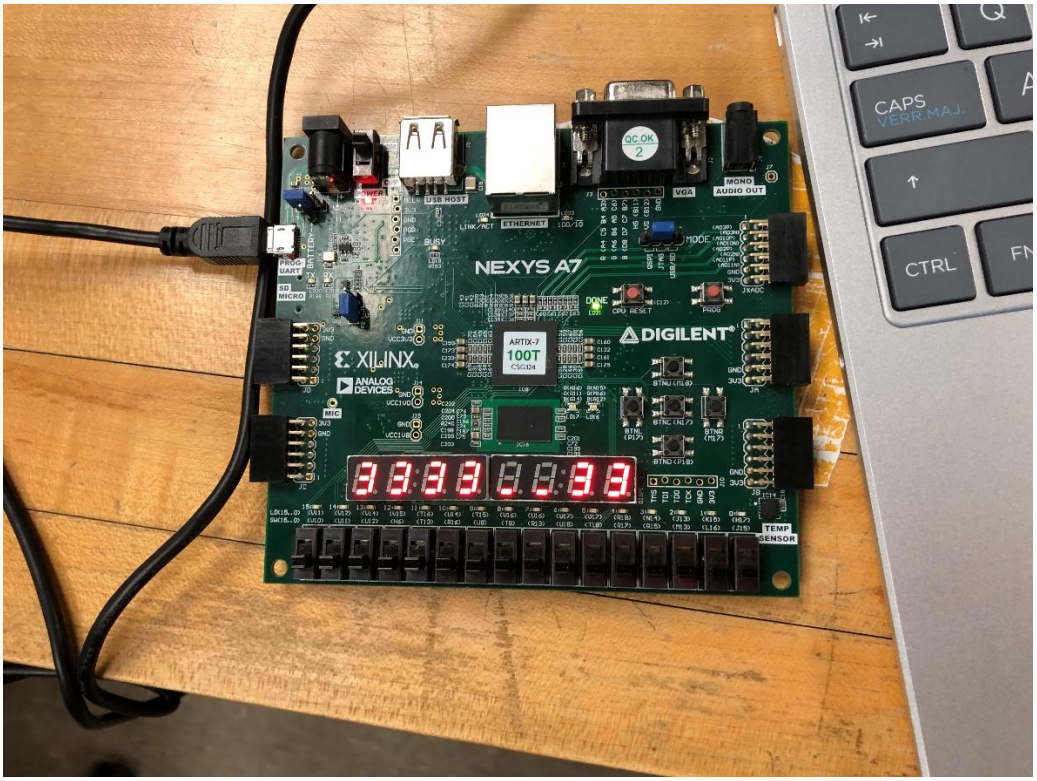


Figure 17: FPGA on second floor with no other floors selected



: FPGA on third floor with no other floors selected (Honors)

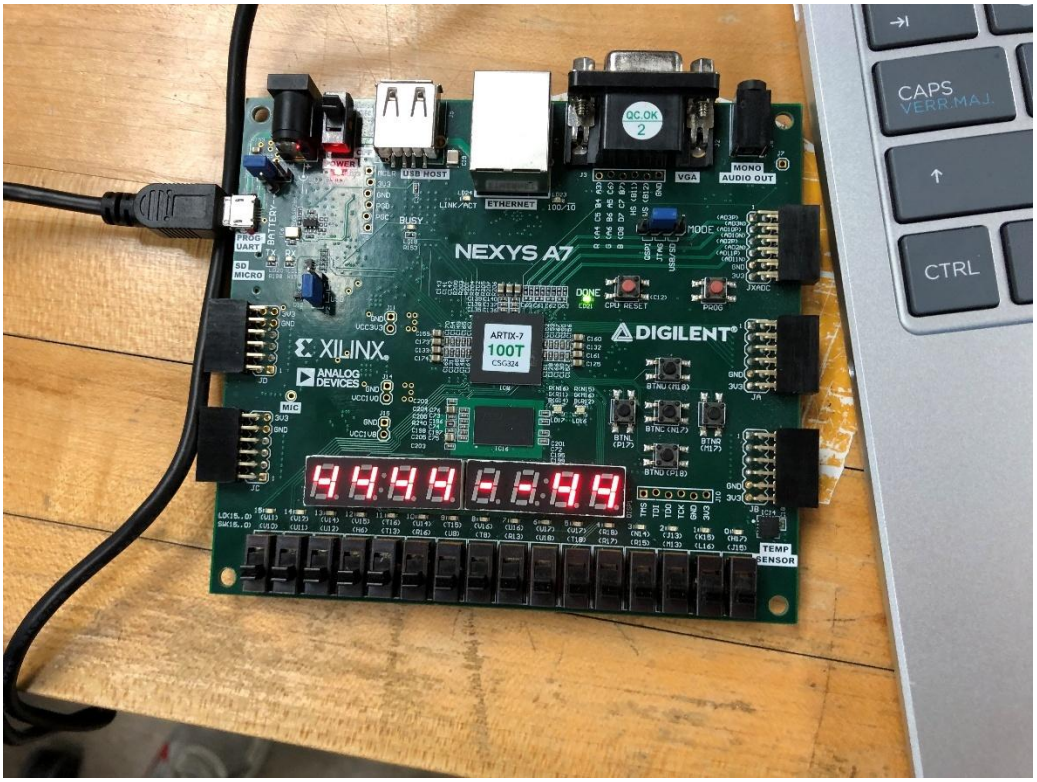


Figure 18: FPGA on fourth floor with no other floors selected

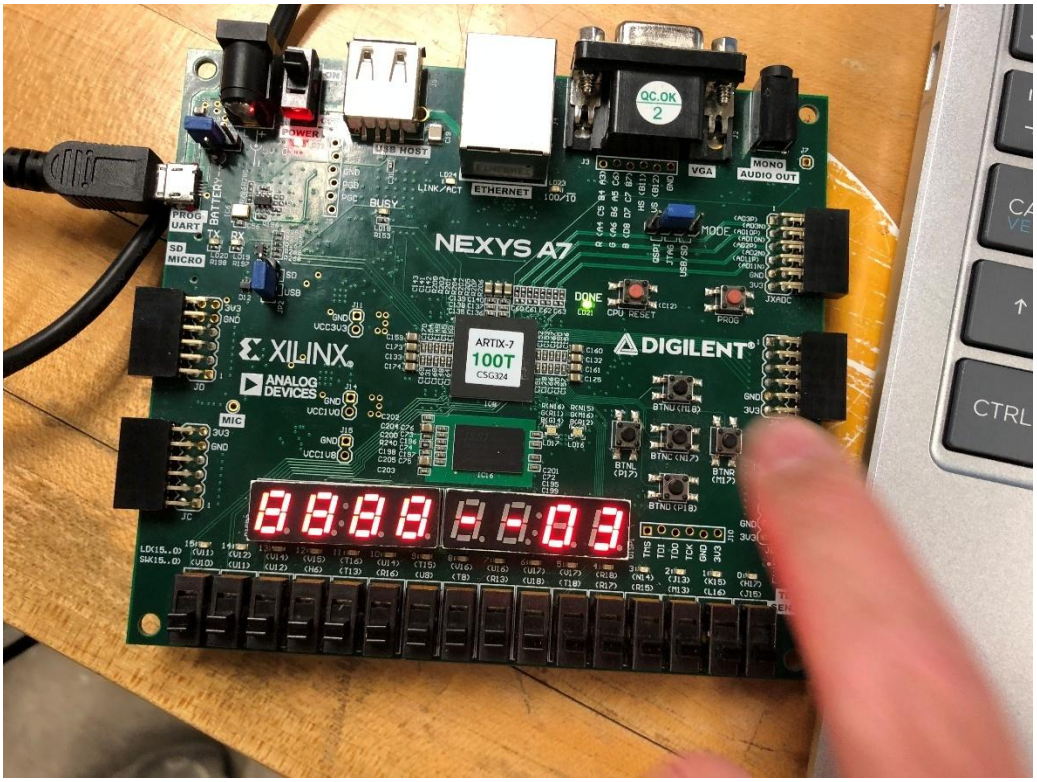


Figure 19: FPGA on ground floor with the third floor just selected

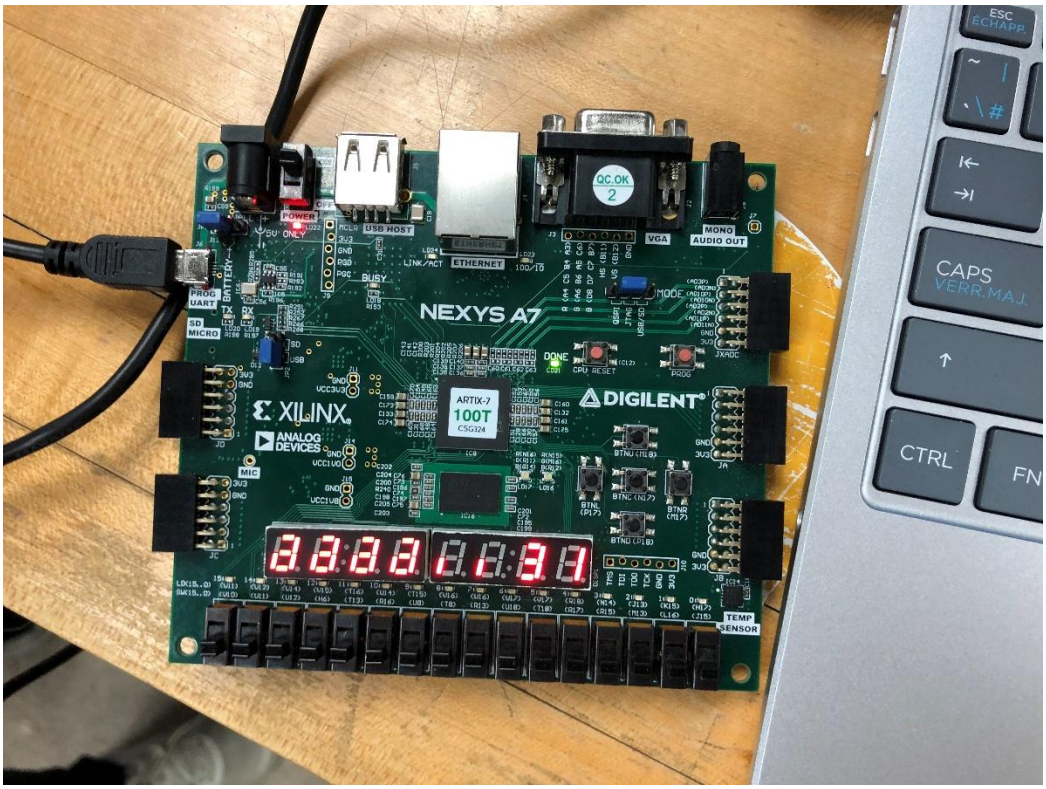


Figure 20: FPGA on third floor with first floor just selected

The previous images show that the system was successfully implemented on the FPGA Board and was able to accurately perform the project requirements. The system stays on a specific floor if no other floors are selected, as the current state is stored, and can effectively transfer to the desired floor that is selected. For example, this was shown in the scenario when the elevator goes from the ground floor to the third floor, and vice versa when it goes from the third floor back to the first floor, when the respective inputs were selected.

The images also show the results of activating both the reset and emergency states via the input switches. In the case of the reset state, the seven-segment displays were turned off; however, the right-most display was forced to display zero. This is a special output that only

occurs when the reset is flipped. In the scenario of the emergency state, it is vital that the state is very clear to the user. Therefore, whenever this switch is flipped, all the seven-segment displays displayed the letter “E”. This makes it evident that the emergency switch has been flipped on and that appropriate actions should be taken. This feature is crucial for safety measures and quality control within a real elevator, as well as the elevator controller within this project.

Conclusion:

This project has effectively displayed that an elevator controller can be created as a finite state machine and implemented in Verilog. Furthermore, it could be uploaded to the FPGA board, where the inputs and outputs could be visually displayed using the respective corresponding components. In the process of constructing the system, the initial step was the completion of the state diagram, which displays the various states and the inputs and outputs at each step. Once this was completed, the basic finite state machine could be implemented within the Verilog code. This allowed for the creation of the timing diagram to ensure the desired outputs corresponded with the various inputs and the state. Once this was created, separate files of Verilog code were used to create a frequency divider for the clock input, a time-multiplexing unit that allows the seven-segment display outputs to be shown nearly simultaneously, and a constraint file that indicated the inputs and outputs to their corresponding components on the FPGA Board.

Once all the separate files were created, a top module that instantiated all the codes together was utilized to complete the system. After this was done and all errors were debugged, the system was synthesized, implemented, and uploaded as a bitstream file onto the FPGA board. This allowed for the visual system of the elevator controller to be shown on the FPGA board, where the user can control the various inputs, while the corresponding outputs are displayed. Any errors noticed within the FPGA Board were debugged and fixed, so that the system fulfilled all the requirements of a typical elevator controller.

Therefore, through various Verilog codes, hierarchical design within a top module allowed for the desired system to be created. This project is vital as it teaches the importance and foundation of finite state machines and how they can be implemented using Verilog code on hardware, such as the Nexys A7 FPGA Board. Finite state machines are present all around us in society today, as they are utilized in a wide range of applications, including both hardware and software systems. This includes artificial intelligence, game development, and various appliances such as vending machines, traffic lights, and elevators, such as the one this project modelled.